

IcoPro

IcoPro

- `train.py` `--icopro_*` .
- `thinker/learn_actor.py` `SActorLearner` , Ray self-play PPO/V-trace
`. FrameStackedBehavioralDataLoader` , `(prev_actions)`,
`(sequence_starts)` 1-step (rollout) .
- `thinker/learn_model.py` `LearnModel` . . .
- `(icopro_supervised_freq)` Adam , self-play .
- `wandb` `thinker/total_loss, imit/total_loss, icopro/actor/*, icopro/model/*` RL .
- RL IcoPro .
- IcoPro DQfD-style margin + cross-entropy + PvP .
- IcoPro

Loss Details

- **Planner-enhanced score q^* .** IcoPro 1-step rollout ,
`planner` ℓ q^{tree} . λ_{tree} $q^* = \ell + \lambda_{\text{tree}} q^{\text{tree}}$,
 $\lambda_{\text{tree}} = 0$, `planner` .

- **Margin matrix m .** Inside the DQfD objective we build a $B \times |\mathcal{A}|$ matrix with entries $m_{i,a} = \alpha \cdot \mathbf{1}[a \neq a_i^H]$. This means only the expert column stays at zero, while all non-expert actions are padded upward by α . Broadcasting m across q^* effectively raises the bar the rival actions must clear before they can beat the demonstration, which is precisely what the hinge loss evaluates.
- **DQfD margin loss.** For each demonstration (o_i, a_i^H) , the planner-enhanced scores $q_{i,a}^*$ combine logits and tree estimates. The per-sample margin matrix assigns $m_{i,a} = 0$ if $a = a_i^H$ and $m_{i,a} = \alpha$ otherwise, enforcing a gap of at least α between the expert action and competitors. The loss implemented in `thinker/imitation.py` is:

$$L_{\text{margin}} = \frac{1}{B} \sum_{i=1}^B \left[\max_{a \in \mathcal{A}} (q_{i,a}^* + m_{i,a}) - q_{i,a_i^H}^* \right]$$

which causes any action whose margin-adjusted value overtakes the expert to incur a positive penalty.

- **PvP regularizer.** Let $a_i^{\text{agent}} = \arg \max_a \ell_{i,a}$ denote the action preferred by the current policy logits ℓ , and define $q_{i,a}^*$ as above. Internally we build one-hot targets $t^+ = +1$ for the human action and $t^- = -1$ for the policy-preferred action whenever they disagree. The PvP term then pushes the human-aligned score toward the positive target while pulling any mismatched agent preference toward the negative target, effectively performing a calibrated two-way regression between planner-backed values and discrete action identities:

$$L_{\text{PvP}} = \frac{1}{B} \sum_{i=1}^B \left[(q_{i,a_i^H}^* - 1)^2 + \mathbf{1}[a_i^{\text{agent}} \neq a_i^H] (q_{i,a_i^{\text{agent}}}^* + 1)^2 \right]$$

This mirrors the mean-squared-error structure implemented in `thinker/thinker/learn_actor.py`, rewarding agreement between the tree-informed Q-values and the human policy while penalizing confident disagreement.

- **KL regularisers.** self-play $\tilde{\pi}_H$, KL .
1. RL `icopro_actor_kl_coef` $\mathbb{E}_s[\text{KL}(\tilde{\pi}_H(\cdot | s) \parallel \pi_\theta(\cdot | s))]$.
2. IcoPro `icopro_bc_actor_kl_coef` `icopro_model_kl_coef` planner
KL . margin/PvP .

CLI		
π_θ	<code>--preload</code>	Initial Thinker actor policy parameters that seed IcoPro; this policy receives standard RL gradients and the offline corrective gradients described below.
M_ϕ	<code>--preload</code>	World model weights loaded before training; IcoPro actor planner rollout
$\mathcal{D}_L$	<code>--icopro_data_path</code>	Human-labeled trajectory buffer providing observation stacks, previous-action markers, rewards, and done targets for offline corrections.
B	<code>--icopro_batch_size</code>	Mini-batch size sampled from $\mathcal{D}_L$ whenever an IcoPro actor or model update is triggered.
α	<code>--icopro_margin</code>	DQfD large-margin constant that enforces the expert action to be at least α higher in q^* than competing actions.
λ_{margin}	<code>--icopro_margin_coef</code>	Scalar multiplying the DQfD margin loss inside the actor objective to balance against other supervised terms.

CLI		
λ_{CE}	<code>--icopro_ce_coef</code>	Weight on the cross-entropy loss between policy logits and human actions; regulates how aggressively logits match demonstrations.
λ_{PvP}	<code>--icopro_pvp_coef</code>	Coefficient on the policy-versus-policy (PvP) regularizer that constrains human and agent Q-ranks to stay aligned.
λ_{tree}	<code>--icopro_tree_coef</code>	Mix-in factor applied to tree-based Q estimates when constructing q^* for the supervised actor loss.
λ_{model}	<code>--icopro_model_coef</code>	Global weight applied to the sum of model heads (reward, policy, state, done) during the IcoPro model update.
$\lambda_{\text{KL}}^{\text{actor}}$	<code>--icopro_actor_kl_coef</code>	Weight on the KL penalty between the mixed human/self-play prior and the online policy during RL updates.
$\lambda_{\text{KL}}^{\text{bc}}$	<code>--icopro_bc_actor_kl_coef</code>	Weight on the KL penalty applied inside the offline supervised batch so that human actions receive extra probability mass.

CLI		
$\lambda_{\text{KL}}^{\text{model}}$	<code>--icopro_model_kl_coef</code>	Weight on the KL penalty that aligns planner root logits with the human prior during the same offline update.
ρ_{mix}	<code>--icopro_selfplay_mix_ratio</code>	Mixing ratio between the human action prior and the EMA of self-play actions when forming the reference distribution $\tilde{\pi}_H$.
β_{EMA}	<code>--icopro_selfplay_ema</code>	Exponential moving-average coefficient that updates the self-play action histogram.
f	<code>--icopro_supervised_freq</code>	Interval in online updates between IcoPro corrections; only every f -th actor/model step consumes human data.
$\eta_{\text{actor}}, \eta_{\text{model}}$	<code>--icopro_actor_lr, --icopro_model_lr</code>	Learning rates for the dedicated Adam optimizers that apply the actor and model IcoPro gradients, respectively.

Algorithm~1 tracks two counters: c_π counts how many IcoPro actor triggers have fired, and c_M does the same for the model pathway. Incrementing, e.g. $c_\pi \leftarrow c_\pi + 1$, simply records that another trigger occurred; the subsequent modulus check enforces the frequency gate so that only every f -th trigger actually runs an offline batch. The dataset $\mathcal{D}_L$ is the buffered human demonstration store defined earlier, and M_ϕ denotes the world-model parameters that the planner and adapter share. Both symbols appear in the pseudo-code to highlight when

the algorithm pulls batches from human data versus when it refreshes the planner-facing network.

Algorithm 1: IcoPro- Thinker

Input: $\pi_\theta, M_\phi, \mathcal{D}_L, B, \alpha, \lambda_{\text{margin}}, \lambda_{\text{CE}}, \lambda_{\text{PvP}}, \lambda_{\text{tree}}, \lambda_{\text{model}}, f,$
 $\eta_{\text{actor}}, \eta_{\text{model}}$

Init: IcoPro $c_\pi \leftarrow 0$, IcoPro $c_M \leftarrow 0$

1 **for each** *IcoPro actor trigger* **do**

2 $c_\pi \leftarrow c_\pi + 1$;

3 **if** $c_\pi \bmod f \neq 0$ **then** //

4 **continue**;

5 $(o, a^H, r^H, d^H) \sim \mathcal{D}_L$ with size B ;

6 M_ϕ planner ;

7 $(\ell, q^{\text{tree}}) \leftarrow \text{PLANNERROLLOUT}(o, r^H, d^H)$ // ℓ : policy logits, q^{tree} : 1-step

6 $q^* \leftarrow \ell + \lambda_{\text{tree}} q^{\text{tree}}$;

8 $\tilde{\pi}_H \leftarrow \text{MIXPRIOR}(a^H, \hat{\pi}_{\text{self}}, \rho_{\text{mix}})$; // prior + self-play EMA

9 $L_{\text{margin}} \leftarrow \text{DQfD}(q^*, a^H, \alpha)$;

10 $L_{\text{CE}} \leftarrow \text{CrossEntropy}(\ell, a^H)$;

11 $L_{\text{PvP}} \leftarrow \text{PvP}(q^*, \pi_\theta)$;

12 $L_{\text{KL}}^{\text{bc}} \leftarrow \lambda_{\text{KL}}^{\text{bc}} \text{KL}(\tilde{\pi}_H \parallel \sigma(\ell))$;

13 $L_{\text{KL}}^{\text{model}} \leftarrow \lambda_{\text{KL}}^{\text{model}} \text{KL}(\tilde{\pi}_H \parallel \sigma(q^{\text{tree}}))$;

14 $L_\pi^{\text{IcoPro}} \leftarrow \lambda_{\text{margin}} L_{\text{margin}} + \lambda_{\text{CE}} L_{\text{CE}} + \lambda_{\text{PvP}} L_{\text{PvP}} + L_{\text{KL}}^{\text{bc}} + L_{\text{KL}}^{\text{model}}$;

15 $\theta \leftarrow \theta - \eta_{\text{actor}} \nabla_\theta L_\pi^{\text{IcoPro}}$ (clip if needed);

16 : $L_\pi^{\text{IcoPro}},$, ;

17 **for each** *IcoPro model trigger* **do**

18 $c_M \leftarrow c_M + 1$;

19 **if** $c_M \bmod f \neq 0$ **then**

20 **continue**;

21 $(s, s', a, r, d) \sim \mathcal{D}_L$ with size B ;

22 $y \leftarrow M_\phi(s, a, s')$ // forward pass with teacher action seq if y contains

22 reward head **then**

23 $L_r \leftarrow \text{MSE}(y_{\text{reward}}, r)$;

24 **if** y contains policy head **then**

25 $L_p \leftarrow \text{CrossEntropy}(y_{\text{policy}}, a)$;

26 **if** y contains state head **then**

27 $L_s \leftarrow \text{MSE}((1 - d) \odot y_{\text{state}}, (1 - d) \odot s')$;

28 **if** y contains done head **then**

29 $L_d \leftarrow \text{BCE}(y_{\text{done}}, d)$;

30 $L_M^{\text{IcoPro}} \leftarrow \lambda_{\text{model}} (L_r + L_p + L_s + L_d)$;

31 $\phi \leftarrow \phi - \eta_{\text{model}} \nabla_\phi L_M^{\text{IcoPro}}$ (clip if needed);

32 : $L_M^{\text{IcoPro}},$;

33 **return** π_θ, M_ϕ

Thinker (IcoPro)

Algorithm 2: Thinker (IcoPro)

Input: Actor policy π_θ , value head V_ψ , world model M_ϕ , actor replay $\mathcal{B}_{\text{actor}}$, model replay $\mathcal{B}_{\text{model}}$, model update frequency f_{model} , learning rates $\eta_{\text{act}}, \eta_{\text{model}}$

Init: Reset planner state, clear $\mathcal{B}_{\text{actor}}, \mathcal{B}_{\text{model}}$, set step counter $t \leftarrow 0$

```

1 for each actor learner step do
2    $t \leftarrow t + 1$ ;
3   self-play  $\tau$  V-trace  $(\hat{v}, \hat{a})$  ;
4    $\hat{\pi}_{\text{self}} \leftarrow \text{UPDATEEMA}(\hat{\pi}_{\text{self}}, \tau, \beta_{\text{EMA}})$ ;
5    $\tilde{\pi}_H \leftarrow \text{MIXPRIOR}(\text{human prior}, \hat{\pi}_{\text{self}}, \rho_{\text{mix}})$ ;
6    $L_{\text{KL}}^{\text{actor}} \leftarrow \lambda_{\text{KL}}^{\text{actor}} \text{KL}(\tilde{\pi}_H \parallel \pi_\theta(\cdot \mid \tau))$ ;
7    $\theta, \psi \leftarrow \theta, \psi - \eta_{\text{act}} \nabla_{\theta, \psi} (L_{\text{IMPALA}}(\tau, \hat{v}, \hat{a}) + L_{\text{KL}}^{\text{actor}})$  // + prior
   rollouts  $\mathcal{B}_{\text{actor}}, \mathcal{B}_{\text{model}}$  ;
8   if  $t \bmod f_{\text{model}} = 0$  then
9      $(s, a, r, s', d) \sim \mathcal{B}_{\text{model}}$  ;
10     $M_\phi, \phi, L_{\text{model}}$  ;
11     $\phi \leftarrow \phi - \eta_{\text{model}} \nabla_\phi L_{\text{model}}$ ;
12  : / , , ;
13 return  $\pi_\theta, V_\psi, M_\phi$ 

```

- thinker/learn_actor.py:210–thinker/learn_actor.py:410
- thinker/learn_model.py:276–thinker/learn_model.py:448
- thinker/imitation.py:34–thinker/imitation.py:384
- thinker/bc_loader.py:10–thinker/bc_loader.py:160